

RISC-V ISA & Processor Architecture Cheat-sheet

L0 Essence: The essence of the RISC-V architecture is a modular, regular instruction set definition that supports implementations ranging from simple in-order pipelines with bypass forwarding to highly complex out-of-order execution (Tomasulo algorithm, ROB in-order commit) processors, maintaining shared data consistency across multiple cores via cache coherence (MESI) and weak memory order (RVWMO) barrier instructions.

[M1.1] RISC-V Architecture Axioms

RISC-V redefines systems architecture through an open, modular, and minimalist instruction set design: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Modular Extension Scheme}$: Unlike bloated CISC instruction sets, RISC-V centers on a tiny base integer spec (RV32i / RV64i, only 40 instructions) and expands via standardized extension building blocks such as M (multiply/divide), A (atomic), F/D (single/double float), and C (compressed 16-bit instructions). $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Minimalist Load-Store Architecture}$: Only dedicated memory access instructions (e.g., ld/ldu) interact with physical memory; all arithmetic and logic instructions operate strictly on general-purpose registers, decoupling address calculations from execution logic. $\backslash\text{textbf{f3}}.\backslash\text{textbf{b}}\text{Regular Field Layout}$: Instructions align fields (rs1, rs2, rd) in space across R/I/S/B/U/I formats, minimizing decoder hardwiring fan-in/fan-out latency.

[M1.2] General-Purpose Registers and ABI Calling Convention

RISC-V defines a highly aligned register file to streamline compiler code generation: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Register File}$: RV64i provides 32 general-purpose registers (x0-x31). Register x0 is hardwired to zero (Hardware Zero Register); writes to it are discarded, and reads always return 0, facilitating zero-overhead nop, clear, and comparison operations. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{ABI Calling Conventions}$: $\backslash\text{textbullet}\text{ra}$ (x1): Holds the return address (Return Address). $\backslash\text{textbullet}\text{sp}$ (x2): Stack pointer (Stack Pointer), strictly 16-byte aligned. $\backslash\text{textbullet}\text{gp}$ (x3) / tp (x4): Global pointer and thread pointer. $\backslash\text{textbullet}\text{t0-t6}$ (x5-x7, x28-x31): Temporary registers (Caller-saved). $\backslash\text{textbullet}\text{a0-a7}$ (x10-x17): Function arguments and return values (with a0/a1 holding return values, Caller-saved). $\backslash\text{textbullet}\text{s0-s11}$ (x8-x9, x18-x27): Saved registers (Callee-saved, with s0 / fp acting as frame pointer).

[M1.3] Classic Five-Stage Pipeline Architecture

Classic in-order pipelines utilize temporal parallelism to partition instruction lifecycles into overlapping stages: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Pipeline Stages}$: $\backslash\text{textbullet}\text{Instruction Fetch (IF)}$: Fetches the instruction pointed to by the Program Counter (PC) from the Instruction Cache and increments PC by 4. $\backslash\text{textbullet}\text{Instruction Decode (ID)}$: Decodes the opcode, operands, and immediates, and reads source operands from the Register File in parallel. $\backslash\text{textbullet}\text{Execute (EX)}$: The Arithmetic Logic Unit (ALU) performs computation, checks branch jump conditions, or aggregates memory addresses. $\backslash\text{textbullet}\text{WritebackMemory Access (MEM)}$: Performs read/write operations on the Data Cache for memory instructions, while non-memory instructions pass through. $\backslash\text{textbullet}\text{Write-Back (WB)}$: Writes the ALU computation or retrieved Cache data back to the Register File. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Clock-Edge Triggered}$: Inter-stage registers latch data on the rising edge of each CPU clock cycle, pushing clock frequencies to their limits.

[M1.4] Control and Status Registers (CSRs) Access Control

Processor settings, privilege modes, and interrupt configurations are managed through CSRs: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Dedicated Address Space}$: A separate 12-bit address space supports up to 4096 CSRs, physically isolated from general-purpose registers and protected by privilege check limits. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Atomic Read-Modify-Write Instructions}$: RISC-V implements dedicated instructions to modify CSR states without race conditions: $\backslash\text{textbullet}\text{csrrw}$ (CSR Read/Write): Atomically swaps a general-purpose register with a target CSR. $\backslash\text{textbullet}\text{csrrs} / \text{csrrc}$ (Read and Set/Clear): Atomically sets or clears specific bits in a CSR based on a register mask. $\backslash\text{textbullet}\text{csrrwi} / \text{csrrsi} / \text{csrrci}$: Immediate operand variants of atomic operations. $\backslash\text{textbf{f3}}.\backslash\text{textbf{b}}\text{Low-Overhead Monitoring}$: Enables OS code to read system clocks and cycle counters directly without triggering expensive kernel traps.

[M1.5] Multi-Privilege Modes & Trap Handling Paths

RISC-V secures system boundaries via layered hardware privilege rings: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Three Privilege Modes}$: User (U, apps), Supervisor (S, OS kernel), and Machine (M, highest firmware/monitor mode, mandatory for all chips). $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Trap Handling}$: When a lower privilege mode triggers an exception or makes a system call (eca11): $\backslash\text{textbullet}\text{The hardware saves the current PC to the target mode's xepc}$ (e.g., mepc / sepc). $\backslash\text{textbullet}\text{It logs the exception code to xcause and logs helper addresses to xip}$. $\backslash\text{textbullet}\text{The privilege mode is raised, and the PC jumps to the vector entry defined in xtvec}$. $\backslash\text{textbf{f3}}.\backslash\text{textbf{b}}\text{Execution Resumption}$: Once the trap handler completes, the software executes xret (e.g., mret), restoring the PC from xepc and returning to the original privilege level.

[M2.1] Pipeline Hazards and Hardware Stalls

A hazard occurs when subsequent instructions in an overlapping pipeline cannot execute in their designated clock cycles: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Hazard Classifications}$: $\backslash\text{textbullet}\text{Structural Hazards}$: Hardware resource conflicts where multiple instructions request the same block (e.g., simultaneous memory read and write on a single-ported Cache). $\backslash\text{textbullet}\text{Data Hazards}$: Instructions depend on operand values that have not yet been written back by preceding instructions. Types include Read-After-Write (RAW), Write-After-Read (WAR), and Write-After-Write (WAW). In in-order pipelines, only RAW (true dependency) causes stalls. $\backslash\text{textbullet}\text{Control Hazards}$: The next PC address cannot be determined until branch target computations complete. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Hardware Stalls}$: When the ID stage detects a RAW conflict that cannot be resolved via forwarding, the pipeline controller freezes the IF/ID registers and inserts NOP bubbles into the EX stage.

[M2.2] Data Bypass Forwarding Mechanism

Most Read-After-Write (RAW) hazards can be resolved by forwarding values directly to execution units, bypassing register file writeback: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Bypass Paths}$: Multiplexers are added to the ALU inputs. The results of the current ALU calculation (in the EX/MEM pipeline register) or data retrieved from Cache (in the MEM/WB pipeline register) are routed directly to the ALU inputs. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Forwarding Control Logic}$: When the controller detects that a source register in the ID/EX stage (rs1/rs2) matches the destination register (rd) of an active instruction in EX/MEM or MEM/WB, it switches the input multiplexer to use the forwarded value. $\backslash\text{textbf{f3}}.\backslash\text{textbf{b}}\text{Load-Use Limit}$: If a Load instruction is immediately followed by an ALU instruction that requires its loaded value, the data is not ready until the end of the MEM stage. The hardware must insert a 1-cycle stall.

[M2.3] Control Hazards and Pipeline Flushes

Branch instructions introduce control flow uncertainty into deeply pipelined microarchitectures: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Pipeline Flush}$: In a standard five-stage pipeline, branch targets and directions are resolved in the EX stage (stage 3). If a branch is taken, the IF stage has already fetched two invalid instructions. The controller must zero out the control registers in the IF/ID and ID/EX stages, transforming the instructions into NOPs (flushing), which incurs a 2-cycle penalty. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Branch Delay Slot}$: To eliminate this penalty, legacy architectures like MIPS executed the instruction immediately following the branch. RISC-V rejects delay slots because they complicate out-of-order execution, relying instead on dynamic branch prediction.

[M2.4] Branch Predictors and Speculative Execution

Branch predictors guess branch directions during the fetch stage to minimize control hazard penalties: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Branch Target Buffer (BTB)}$: A hardware cache queried during the IF stage. If the current PC matches an entry, the BTB outputs the previously recorded target PC, bypassing decoding and branch offset addition. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Branch History Table (BHT)}$: An array of 2-bit saturating counters. States are: 00 (strongly not taken), 01 (weakly not taken), 10 (weakly taken), and 11 (strongly taken). The counter increments or decrements based on the actual branch outcome, preventing single loop-exit transitions from misaligning predictions. $\backslash\text{textbf{f3}}.\backslash\text{textbf{b}}\text{gshare Predictor}$: XORs the Global Branch History Register (GBHR) with the PC to index the BHT, capturing conditional correlation across multiple branches.

[M2.5] Precise Exception Handling and Rollback

Processors must protect register states during exceptions (e.g., page faults) or external interrupts: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Precise Exception Definition}$: $\backslash\text{textbullet}\text{All instructions preceding the exception instruction } I_P \text{ must execute and commit their register changes}$. $\backslash\text{textbullet}\text{The exception instruction } I_P \text{ and all subsequent instructions in the pipeline must be flushed, leaving no trace in the register file}$. $\backslash\text{textbullet}\text{The PC is saved to xepc}$ (e.g., mepc), pointing back to I_P for execution resumption after the trap handler finishes. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Hardware Flags}$: Exceptions are flagged in registers at each stage and propagated down the pipeline. The exception is only handled when the offending instruction reaches the writeback/commit boundary, ensuring precise state capture.

[M3.1] Out-of-Order Execution and the Tomasulo Algorithm

In-order cores halt execution for long-latency instructions like Cache misses, stalling unrelated downstream instructions. Out-of-order execution resolves this: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Tomasulo Algorithm}$: Tracks operand dependencies using Reservation Stations (RS) and broadcasts results over a Common Data Bus (CDB). $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Reservation Stations (RS)}$: Decoded instructions enter an available RS slot. If an operand is ready in the register file, it is copied to the RS; if not, the RS records the tag of the virtual execution unit (or physical register) that will produce the value. $\backslash\text{textbf{f3}}.\backslash\text{textbf{b}}\text{CDB Broadcast}$: When an execution unit completes a calculation, it broadcasts the result and its tag over the CDB. RS slots listen to the bus, grabbing values matching their pending tags. Once all operands are ready, the instruction is dispatched for execution.

[M3.2] Register Renaming Mechanism

Register renaming eliminates false data dependencies to enable out-of-order execution: $\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Eliminating False Dependencies}$: WAR (Write-After-Read) and WAW (Write-After-Write) hazards are names conflicts caused by compiler register reuse, not true dataflow. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Physical Register File (PRF) and Map Table}$: $\backslash\text{textbullet}\text{The hardware includes a large PRF}$ (e.g., 128 physical registers) mapped to 32 architectural registers (x0-x31). $\backslash\text{textbullet}\text{The renaming process}$: For every write destination (rd), the decoder pulls an unused physical register P_k from the Free List and maps the logical register rd to P_k in the Map Table. Subsequent instructions reading this register are redirected to P_k . $\backslash\text{textbf{f3}}.\backslash\text{textbf{b}}\text{Dependency Decoupling}$: Renaming breaks name conflicts, allowing independent instruction streams to execute concurrently.

L1 Logical Framework

$\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Modular Specs \& Privilege Control}$: RISC-V defines structured general-purpose registers and a CSR-controlled multi-privilege trap handling channel, enabling hardware-level precise capture of instructions and exceptions. $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Lockstep Stalls \& Branch Speculation}$: The five-stage pipeline resolves data dependencies using bypass forwarding and reduces control flow jump penalties via dynamic branch predictors (BHT/BTB). $\backslash\text{textbf{f3}}.\backslash\text{textbf{b}}\text{Renaming, Reordering \& In-Order Commit}$: Out-of-order execution resolves false dependencies via Tomasulo reservation stations and register renaming, restoring execution order at the commit stage through a Reorder Buffer (ROB). $\backslash\text{textbf{f4}}.\backslash\text{textbf{b}}\text{Snooping Coherence \& Weak Ordering}$: Cache lines are kept coherent across cores via MESI snooping protocols, and RVWMO weak memory order enforces synchronization using FENCE barriers and LR/SC atomic primitives.

L2 Data Flow Topology

$\backslash\text{textbullet}\text{PC fetch} \rightarrow \text{Instruction Cache read} \rightarrow \text{Decode (ID)} \rightarrow \text{Register Rename (Map Table lookup)} \rightarrow \text{Dispatch to Reservation Station (RS)} \rightarrow \text{Wakeup \& Select} \rightarrow \text{Read operands from Physical Register File (PRF)} \rightarrow \text{ALU Execute} \rightarrow \text{Data Hazard? No, Bypass Forwarding activated} \rightarrow \text{Memory Load/Store Buffer queue} \rightarrow \text{L1 Data Cache look up} \rightarrow \text{Cache Miss? Yes} \rightarrow \text{TLB virtual to physical walk} \rightarrow \text{MESI Bus Request} \rightarrow \text{State Invalid} \rightarrow \text{Broadcast Read Invalid} \rightarrow \text{Fill L1 Cache Line} \rightarrow \text{State Modified} \rightarrow \text{Store-to-Load Forwarding} \rightarrow \text{Writeback result to ROB} \rightarrow \text{ROB in-order commit} \rightarrow \text{Commit to Architectural Register File}$.

Trade-off Matrix: Pipeline Overhead & Accuracy

$\backslash\text{textbf{f1}}.\backslash\text{textbf{b}}\text{Pipeline Scheduling Policy}$: In-order execution with lockstep stalls (In-order Execution & Stall) $\rightarrow$ Out-of-order execution with Tomasulo algorithm (Out-of-order Execution & RS/ROB) [In-order cores require simple bypass forwarding to resolve RAW hazards $\rightarrow$ but stall on Cache misses. Out-of-order cores bypass false dependencies via reservation stations and renaming to exploit instruction-level parallelism (ILP) $\rightarrow$ but require complex Map Tables, Issue Queues, and ROBBS, increasing silicon area and power consumption [In-order pipelines are suited for low-power embedded IoT; out-of-order designs are preferred for high-performance servers].] $\backslash\text{textbf{f2}}.\backslash\text{textbf{b}}\text{Register File \& Renaming}$: Direct architectural register access (32 registers, direct addressing) $\rightarrow$ Physical register renaming (Map Table & 128+ physical register file) [Direct register reads require no renaming logic, completing in 1 cycle with minimal latency $\rightarrow$ but restrict out-of-order scheduling. Renaming resolves WAR/WAW conflicts via spatial registers $\rightarrow$ but Map Table queries and Free List management add silicon area and timing constraints [Use direct register access for low-latency, low-power cores; use renaming for high-IPC out-of-order cores].] $\backslash\text{textbf{f3}}.\backslash\text{textbf{b}}\text{Branch Prediction Control}$: Static or simple 2-bit counter prediction (Static/2-bit Predictor) $\rightarrow$ Global history correlation with gshare (Global History & gshare Predictor) [2-bit counters require minimal hardware overhead $\rightarrow$ but suffer high misprediction rates in complex loops, causing pipeline flushes. gshare tracks execution history to achieve >95% accuracy $\rightarrow$ but the history logic and XOR gates add fetch-stage latency [Use 2-bit branch predictors for short in-order pipelines; use gshare or TAGE predictors for deep out-of-order pipelines].] $\backslash\text{textbf{f4}}.\backslash\text{textbf{b}}\text{Memory Sync Model}$: Write-through or write-back L1 caches (Write-Through / Write-Back L1) $\rightarrow$ Store Buffer with Store-to-Load Forwarding (Store Buffer & Forwarding) [In-order memory access requires no Store Buffers and simplifies address checking $\rightarrow$ but cannot hide memory latency. Out-of-order memory access buffers stores to isolate cache before commit and forwards values to loads, hiding latency $\rightarrow$ but requires complex load-queue collision checks to prevent data corruption [Deploy forwarding for data-intensive processing cores; use simple write-back caches for standard MCUs].]

[M3.3] Instruction Dispatch and Issue Queue Scheduling

Renamed instructions are placed in the Issue Queue (IQ) or Reservation Stations to wait for dispatch: `\\textbf{f1}\\textbf{WakeUp-Select Loop}`: `\\textbullet\\textbf{WakeUp Phase}`: In each cycle, the CDB broadcasts completed physical tags. The IQ matches these tags against the source operands of pending instructions, marking ready operands. `\\textbullet\\textbf{Select Phase}`: The scheduler picks ready instructions based on priority (e.g., Oldest First) and dispatches them to execution units. `\\textbf{f2}\\textbf{Critical Path Bottleneck}`: The Wakeup-Select loop is a tight feedback loop. Its gate delay scales quadratically with dispatch width (e.g., 4-wide, 8-wide), representing a major limitation on CPU clock speeds.

[M3.4] Reorder Buffer (ROB) and In-Order Commit

Out-of-order execution writes results to registers out of order, which complicates precise exceptions and branch misprediction recovery. The ROB restores program order: `\\textbf{f1}\\textbf{In-Order Commit}`: `\\textbullet\\textbf{As instructions decode, they are allocated entries in the ROB in program order (using a FIFO queue)}`. `\\textbullet\\textbf{When an instruction executes, its result is written to its ROB entry and marked ready, but \\textbf{f1} does not modify the architectural register file (ARF)}`. `\\textbullet\\textbf{Only when the instruction at the head of the ROB (the oldest instruction) is marked ready does the processor commit it, writing the result to the ARF}`. `\\textbf{f2}\\textbf{Load Queue Recovery}`: If a branch misprediction or exception occurs at the ROB head, the processor flushes all subsequent entries in the queue and rolls back the Map Table.

[M3.5] Memory Disambiguation and Store-to-Load Forwarding

Memory instructions can have overlapping addresses, meaning their execution order must be strictly managed: `\\textbf{f1}\\textbf{Store Buffer Isolation}`: To prevent branch mispredictions from corrupting memory, Store instructions write their target addresses and data to a Store Buffer instead of Cache. They only write to Cache at the commit stage. `\\textbf{f2}\\textbf{Store-to-Load Forwarding}`: When a Load executes, its physical address is matched against the Store Buffer and Cache in parallel. If a pending Store matches the Load address, the data is forwarded directly from the Store Buffer to the Load, bypassing Cache. `\\textbf{f3}\\textbf{Load Queue Verification}`: If a Load executes out of order and reads stale data before a preceding conflicting Store address is calculated, the Load Queue flags the conflict and flushes the pipeline.

[M4.1] Cache Set-Associative Structure

Cache architectures exploit program locality to bridge the latency gap between CPU execution and main memory: `\\textbf{f1}\\textbf{Set-Associative Cache}`: Caches are partitioned into S sets, each containing N cache lines (or ways, e.g., 4-way/8-way). `\\textbf{f2}\\textbf{Physical Address Parsing}`: Addresses are parsed into three fields: `\\textbullet\\textbf{Offset}`: Local bytes within a cache line (e.g., 64-byte width). `\\textbullet\\textbf{Index}`: Selects the target Set. `\\textbullet\\textbf{Tag}`: Holds the physical page frame number. `\\textbf{f3}\\textbf{Hit Verification}`: The MMU hashes the address Index to select a Set and compares the physical Tag against all N ways in parallel. If a matching tag is valid, the cache hit is resolved. `\\textbf{f4}\\textbf{Replacement Policy}`: If a set is full, the cache controller evicts a line using algorithms like Least Recently Used (LRU).

[M4.2] Cache Write Policies and Coherence Bottlenecks

Cache write strategies dictate data sync policies and balance bus traffic overhead: `\\textbf{f1}\\textbf{Write Strategies}`: `\\textbullet\\textbf{Write-Through}`: Every write operation updates both the cache line and main memory. While simple and safe, it consumes bus bandwidth and stalls execution on memory writes. `\\textbullet\\textbf{Write-Back}`: Writes update only the local L1 Cache, marking the line as dirty (Dirty bit = 1). The line is only written back to main memory when it is evicted. `\\textbf{f2}\\textbf{Coherence Conflicts}`: In multi-core systems with private L1 Caches, if Core 0 modifies variable X in its L1 and Core 1 reads its own stale copy of X from its private L1, it causes cache inconsistency.

[M4.3] MESI Cache Coherence Protocol

The MESI protocol maintains cache coherence across cores via bus snooping: `\\textbf{f1}\\textbf{Four Cache States}`: `\\textbullet\\textbf{Modified (M)}`: The line is dirty. It exists only in the current core's cache and contains data that differs from main memory. `\\textbullet\\textbf{Exclusive (E)}`: The line matches main memory and exists only in the current core's cache. `\\textbullet\\textbf{Shared (S)}`: The line matches main memory and may exist in other cores' caches. `\\textbullet\\textbf{Invalid (I)}`: The line's data is stale and unreadable. `\\textbf{f2}\\textbf{Bus Snooping}`: When Core 0 writes to a Shared line, it broadcasts an invalidate signal (BusUpgrade / BusRdX) on the bus. The snoop controllers of other cores intercept this signal and mark their local copies as Invalid.

[M4.4] Address Translation and TLB Miss Handling

Virtual-to-physical address translation requires hardware acceleration to minimize performance penalties: `\\textbf{f1}\\textbf{Page Table Walker (PTW)}`: In SV39 (three-level page tables), translating an address requires querying three levels of page tables in memory, which adds translation overhead to memory operations. `\\textbf{f2}\\textbf{Translation Lookaside Buffer (TLB)}`: A high-speed cache inside the MMU that stores recently translated virtual-to-physical address mappings. `\\textbf{f3}\\textbf{PTW Miss Walking}`: When a TLB miss occurs, the PTW walks the page tables in memory starting from the root page table (in the `satp` register). If a Page Table Entry (PTE) is invalid or violates read/write/execute permissions, the PTW triggers a Page Fault Trap; otherwise, it loads the translation into the TLB.

[M5.1] Memory Consistency Models

Memory consistency models define the execution order and visibility of reads and writes across different memory locations: `\\textbf{f1}\\textbf{Coherence vs. Consistency}`: Cache coherence ensures multiple copies of a single memory location match; memory consistency defines order rules across distinct memory locations. `\\textbf{f2}\\textbf{Sequential Consistency (SC)}`: SC requires all cores to see a single, globally interleaved sequence of reads and writes that matches the Program Order (PO) of each individual core. `\\textbf{f3}\\textbf{Ordering Trade-offs}`: To maximize performance, processors use write buffers and out-of-order execution, which violates SC. Maintaining SC would require stalling execution on every memory access. Modern architectures resolve this using weak memory models.

[M5.2] RISC-V Weak Memory Order (RVWMO)

RISC-V adopts a relaxed memory model (RVWMO) to allow aggressive pipeline optimizations: `\\textbf{f1}\\textbf{Instruction Reordering}`: RVWMO allows the processor to reorder memory accesses to different addresses, provided it does not violate single-thread data or control dependencies: `\\textbullet\\textbf{Read-Read, Read-Write, Write-Read}`. `\\textbf{f2}\\textbf{Write Buffer Effects}`: Writes by Thread A may sit in its local Store Buffer before being committed to main memory, making them temporarily invisible to Thread B. `\\textbf{f3}\\textbf{Software Synchronization}`: This model shifts the responsibility of ordering memory accesses to software, which must use explicit barrier instructions.

[M5.3] FENCE and FENCE.I Synchronization

Software inserts barrier instructions to enforce memory ordering when required by application logic: `\\textbf{f1}\\textbf{FENCE Instruction}`: Syntax is `fence predecessor, successor`, where predecessor and successor specify combinations of memory operations: 1 (input/read), 0 (output/write), R (read), and W (write). E.g.: `fence r, rw` Enforces that all reads preceding the fence must complete before any reads or writes following the fence can execute. `\\textbf{f2}\\textbf{FENCE.I Instruction}`: Synchronizes the Instruction Cache (I-Cache) and Data Cache (D-Cache). For self-modifying code (e.g., JIT compilers writing code to data pages), executing `fence.i` flushes the I-Cache to ensure subsequent fetches load the newly written instructions from memory.

[M5.4] Atomic Memory Operations (AMO)

Atomic operations provide hardware-level primitives to build mutual exclusion locks and lock-free queues: `\\textbf{f1}\\textbf{AMO Instructions}`: Syntax is `amoadd w rd, rs2, (rs1)`. These instructions perform a read-modify-write operation directly in memory or cache without loading data into general-purpose registers. `\\textbf{f2}\\textbf{Atomic Execution Paths}`: `\\textbullet\\textbf{Cache Hit}`: If the target cache line is Modified (M), the core updates it in its private L1 Cache while locking the line's MESI state, preventing other cores from modifying it. `\\textbullet\\textbf{Cache Miss}`: The bus controller locks the bus, preventing other cores from accessing memory until the read-modify-write operation completes.

[M5.5] Load-Reserved / Store-Conditional (LR/SC)

LR/SC instructions provide an alternative primitive to construct atomic operations (like Compare-And-Swap): `\\textbf{f1}\\textbf{LR rd, (rs1) (Load-Reserved)}`: Loads a value from address `rs1` into `rd` and registers a reservation on the target address in the core's hardware monitor. `\\textbf{f2}\\textbf{SC w rd, rs2, (rs1) (Store-Conditional)}`: Conditionally writes `rs2` to address `rs1`. `\\textbullet\\textbf{Success Rule}`: The store succeeds if no other core has written to the reserved address since the `lr, w` instruction, and no interrupts or context switches have occurred. It writes 0 to `rd`. `\\textbullet\\textbf{Failure Rule}`: If the reservation is lost, the store fails, and the instruction writes a non-zero error code to `rd` without updating memory. `\\textbf{f3}\\textbf{Optimistic Concurrency}`: Provides a lock-free mechanism that avoids the ABA problem.

[M6.1] RISC-V Vector Extension (RVV) Axioms

Vector extensions accelerated data-parallel workloads (e.g., AI inference and image processing): `\\textbf{f1}\\textbf{Vector Length Agnostic (VLA)}`: Unlike SIMD architectures (e.g., Intel AVX-512, ARM NEON) that hardcode register widths in instructions, RVV instructions do not specify vector widths. Width is determined by the hardware (VLEN, e.g., 128 to 65536 bits), ensuring binary compatibility across chips. `\\textbf{f2}\\textbf{Register File}`: RVV defines 32 vector registers (`v0-v31`). `\\textbf{f3}\\textbf{Portability}`: The same RVV binary can run on a high-throughput vector supercomputer or a low-power IoT SoC, adapting to the target hardware's vector width.

[M6.2] Vector Configuration and Execution Modes

RVV allows software to configure vector parameters dynamically to optimize execution: `\\textbf{f1}\\textbf{vsetvli Instruction}`: Configures vector execution parameters for a loop: `\\textbullet\\textbf{SEW (Selected Element Width)}`: Sets the width of individual elements (8, 16, 32, or 64 bits). `\\textbullet\\textbf{LMUL (Vector Register Grouping)}`: Groups multiple vector registers into single logical registers (e.g., LMUL=8 groups `v0-v7` to increase instruction throughput). `\\textbf{f2}\\textbf{Vector Length Control (v2)}`: The hardware calculates the maximum elements processed per instruction (`v1`) based on SEW, LMUL, and VLEN. The loop increments by `v1` on each iteration.

[M6.3] Hardware Performance Monitors and Debugging

On-chip monitoring and debugging primitives are essential for microarchitectural tuning: `\\textbf{f1}\\textbf{Hardware Performance Monitors (HPM)}`: Read-only registers in the CSR space that track core events. The three base counters are: `\\textbullet\\textbf{cyc1e}`: Counts CPU clock cycles since reset. `\\textbullet\\textbf{time}`: Tracks real-time clock cycles. `\\textbullet\\textbf{instret}` (Instructions Retired): Counts committed instructions, excluding flushed instructions. `\\textbf{f2}\\textbf{CPI Calculation}`: Measuring `cyc1e` and `instret` across code segments yields the Cycles Per Instruction (CPI). `\\textbf{f3}\\textbf{Hardware Debugging}`: The JTAG interface provides access to TAP controllers for boundary scans, breakpoints, register inspection, and single-step execution.

[M6.4] Spike and QEMU Instruction Set Simulators

Before silicon tape-out, software stack development and ISA verification rely on simulators: `\\textbf{f1}\\textbf{Spike Simulator}`: The official golden model simulator. It interprets RISC-V instructions sequentially, maintaining precise CSR, page table, and privilege states for ISA verification. `\\textbf{f2}\\textbf{QEMU Dynamic Translation (TCG)}`: Spike is accurate but slow. QEMU uses JIT compilation (Tiny Code Generator) to translate RISC-V instruction blocks into host instructions (e.g., x86_64), running fast enough to boot Linux and run large software packages.

Zone T1: RISC-V Assembly & CSR Operations

`crrw t0, mepc, x0` : Atomic Read/Write. Reads the Machine Exception Program Counter

`mepc` into register `t0` and writes `x0` (0) to `mepc` (reading and clearing the CSR). `\\csrrs t1, mstatus, t2` : Atomic Read and Set. Reads the Machine Status register `mstatus` into `t1` and sets bits in `mstatus` that are high in `t2` (commonly used to enable global interrupts). `\\lr. w x5, (x10)` : Load-Reserved. Loads a 4-byte word from the memory address in `x10` into `x5` and registers a reservation on that address in the core's hardware monitor. `\\sc. w x6, x7, (x10)`

: Store-Conditional. Writes `x7` to address `x10` if the reservation on `x10` is still valid. On success, writes 0 to `x6`; on failure, discards the store and writes a non-zero error code. `\\vsetvli t0, a0, e32, m8, ta, ma` : Vector Configuration. Configures vector parameters (32-bit element width `e32`, register grouping factor `m8`) for a maximum elements count `a0`, returning the active vector length to `t0`.

Zone T2: Exceptions & Trap Codes

When a trap occurs, the Machine Cause register `meause` (or `scause` in Supervisor mode) records

the exception code. Below is a dictionary of core Exception Codes: `\\textbf{Exception Code} = 0` (Instruction Address Misaligned) : Instruction fetch address misalignment. Triggered if the PC jumps to an address that is not aligned to a 2/4-byte boundary. `\\textbf{Exception Code} = 2` (Illegal Instruction) : Decodes an invalid opcode, or attempts to execute a privileged instruction at an insufficient privilege level (e.g., executing `mret` in User mode). `\\textbf{Exception Code} = 3` (Breakpoint) : Triggered by the `ebreak` instruction, commonly used by debuggers to halt execution. `\\textbf{Exception Code} = 5` (Load Address Misaligned) / `6` (Store/AMO Address Misaligned) : Memory address misalignment. Triggered when executing a memory load or store to an unaligned address on cores that do not support unaligned accesses. `\\textbf{Exception Code} = 8` (Environment Call from U-mode) : User-mode system call. Executing `ecall` in User mode shifts execution to Supervisor mode (OS kernel) to handle system services. `\\textbf{Exception Code} = 12` (Instruction Page Fault) / `13` (Load Page Fault) / `15` (Store/AMO Page Fault) : Page fault exceptions. Triggered when the PTW walks page tables for translation and encounters an invalid PTE (Valid=0) or a read/write/execute permission violation.